International Journal of Neural Systems, Vol. 0, No. 0 (2005) 1–9

A Machine Learning-based Pipeline to Detect Conjunctivitis from Images Taken with Mobile Camera

Akash Kotal, Sidhant Singh
Department of Computer Science & Engineering,
Techno International New Town,
Kolkata, 700156, India

Aayusman Chatterjee
Department of Computer Science & Engineering,
Institute of Engineering & Management, Kolkata, India

Nilanjana Dutta Roy
Department of Computer Science & Engineering,
Techno International New Town Kolkata, India

Mufti Mahmud
Department of Computer Science, Nottingham Trent University,
Clifton, NG11 8NS Nottingham, UK

One of the prevalent and contagious eye conditions that affect the conjunctiva of the human eye is Conjunctivitis. Eye drops and other medications can be used to treat both the bacterial and viral forms of it. The condition must be identified at an early stage in order to understand how it is related to other illnesses, including SARS-CoV-2. One such alternative that works well for the initial screening of Conjunctivitis is any mobile application. A strong deep-learning model is needed to do this task successfully. In this study, following a comparison with seven other deep learning models, the best model is chosen for the mobile application for functioning. A comparative analysis of all the selected models is also presented here. The study reports the commendable performance of the best model with 98.4% accuracy for binary classification.
The source code for this work can be found at: *https://github.com/sky01green/icondet.*

Keywords: Conjunctivitis; Binary classification; Deep Learning models; Comparative analysis.

1. Introduction

Eye infections caused by bacteria and viruses affect people throughout the year on average. Home cures or some over-the-counter eye drugs can be used to treat most conditions unless they are very serious. People must visit a doctor for additional therapy if the problem is complicated. But in some urgent cases, visiting a doctor becomes difficult, particularly when there is a pandemic. Therefore, initial screening of eye diseases through mobile/web applications plays an important role in determining the kind and severity of the disease. One such smartphone application that excels at this preliminary screening is *i*ConDetV2. The second version of it was released in 2022. The first phase of this Android application is binary classification, and the second is severity detection. Here, an image of an eye is uploaded using the app after being taken using a mobile phone. Then, the results are made known through the application in a very short time. People can save time by using *i*ConDetV2 as an initial screening mobile application. It becomes easy for ophthalmologists also to diagnose the disease and begin treatment. Therefore, a robust deep-learning model is crucial for updating and improving the released Android application with a new version.

Conjunctivitis, a typically contagious and seasonal eye condition, can occur at any time of the year.[1] It affects the conjunctiva, the eye's transparent and thin outer layer. Pink eye disease or conjunctivitis is the term used to describe any irritation in the conjunctiva brought on by bacteria or viruses. It can be identified by a few symptoms including red eyes and inflammation, irritation, swelling, and itching. The two main causes of conjunctivitis are viruses and bacteria. Other elements, such as chemical exposure and allergy diseases, may also contribute to discomfort. Eye drops containing steroids are used to treat viral conjunctivitis, which spreads by tears and droplets.[2] Antibiotics are frequently used to treat the bacterial kind of it.[?] A cool compress and a saline water flush may help ease discomfort in some cases of allergic and chemical Conjunctivitis.[3]

Interestingly, the pink eye has been reported in certain Covid-19 positive patients.[4] According to a few studies, patients with conjunctivitis and new coronavirus pneumonia may have Covid-19 in their tears and conjunctival secretions,[5,6] Only eye irritations, swollen and red eyes, and photophobia have been used to diagnose severe Covid-19 infections in a small number of asymptomatic individuals. There have been few cases where the sole signs of a coronavirus infection were conjunctivitis and translucent serous discharges.[5] Therefore, in an appropriate clinical setting, ophthalmologists may play a crucial role in identifying Covid-19 infections during the pandemic.[7] Reverse Transcriptase Polymerase Chain Reaction (RT-PCR) analysis of tears revealed the presence of the new coronavirus in a few instances. Although individuals infected with the coronavirus typically exhibit respiratory discomfort, a common symptom like conjunctivitis may also be seen.[8] In a different case report, coronavirus was found in ocular fluids even two weeks after nasopharyngeal RT-PCR results were negative.[9] However, more research is necessary to determine whether the virus can spread the disease through ocular secretions because the virus' ability to do so is still unknown. Therefore, it is crucial to take into account and further investigate the issue of transmission by contact with the conjunctiva,[8,10] Looking at the current situation, it would be ideal to improve the AI-based, user-friendly mobile application for initial screening. Accordingly, we concentrated on upgrading the AI-based Android application to identify conjunctivitis in people's eyes based on the information we obtained. In order to do this, a thorough analysis of seven distinct deep-learning models was conducted in this study. The finest one is selected from these and will eventually be incorporated into the application in the future.

*i*ConDetV3 will be created primarily as a practical method to identify conjunctivitis. For the coronavirus disease, it will serve as a preliminary screening and preventive measure. It will be able to carry out the necessary pre-processing on ocular pictures taken with a mobile device in the future. From this study, the selected fittest deep-learning model will be used for the further development of this Android application.

The main contribution of the work is described in the following points:

- A comparative analysis of the most effective deep learning models was conducted, and the best option with the best performance was selected for the application suitable for Andriod devices.
- One of the unique aspects of this work is implementing the Unet architecture for segmentation.

The overall description of the system is provided in Fig. 1. The structure of the architecture is as follows:

- The user chooses an eye image either using the mobile application's camera feature or from the local storage (such as the gallery).
- To separate the unimportant portion of the eye image, the hand-eraser tool and rectangle cropper are used during image processing.
- The cropped image is saved locally before being chosen as input for the machine learning model that was trained using a dataset of cropped eye photos.

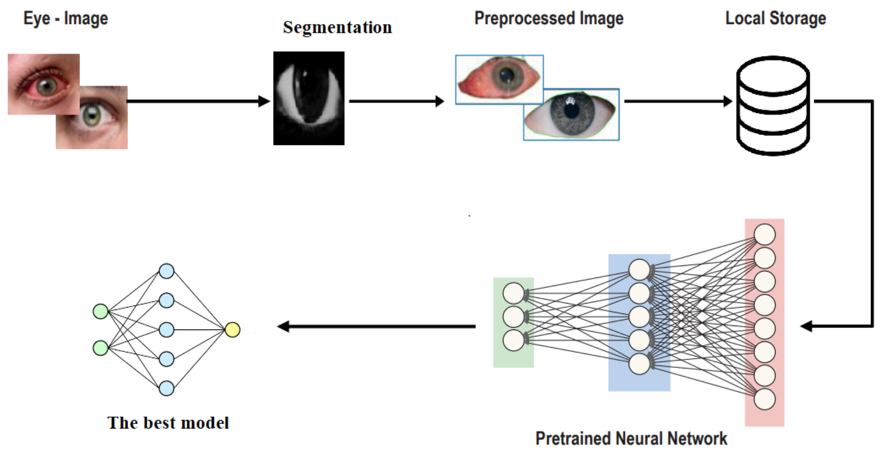


Figure 1. The overall description of the system.

2. Methodology

Since there was no particular eye dataset available for this study, the dataset used has been constructed from scratch by consulting different hospitals in Kolkata itself. From this dataset, two distinct datasets of eyes have been created, one with healthy images and the other with infected images. For the Unet model, masked images are required for training. Masked images are images where the background is obscured by setting some values of the pixels to zero. The images in the dataset are then masked individually using Labkit tool in the ImageJ software. ImageJ is a software that is used to display, annotate, edit, calibrate, measure, analyze, process, print, and save raster (row and column) image data. For further preprocessing, the images are prepared to be first imported in .jpg and resized to 128 pixels. During this process, the images were also converted from RGB to BGR color format. These resized images are then stored in a numpy array for proper manipulation if needed. Now, masked images are also imported but in .tif format, and similar to the previous images, these masked images are also resized to 128 pixels. Since these are masked images, they are not converted from RGB to BGR color format. So, after resizing, the masked images are also stored in numpy array.

The U-net architecture was invented to resolve the segmentation of biomedical images. U-net does semantic segmentation which is the pixel-based classification where each pixel is classified under a particular class. The U-net architecture has an Encoder and Decoder path. The Encoder is on the left side of the architecture consisting of two 3x3 convolution networks followed by ReLu and 2x2 max pooling layers and at each downsampling step the feature channels are doubled and the spatial dimensions are halved. For the purpose of classification, some deep learning models such as VGG16, ResNet50, InceptionV3 have been used in this study. The dataset used in this project is categorized into three parts: training, testing, and validation. Initialization of the data preparation is done by first importing the dataset. Then, the images are converted to image arrays. Following this, two arrays are created during the preprocessing stage, containing the values and labels respectively, to be used in the training, testing, and validation stages for the models. By doing so, the necessary datasets are made ready for the subsequent classification process. In order to prepare the images for further operations, this preprocessing step is implemented over all three datasets: training, testing, and validation. In this manner, the preparation of corresponding datasets is completed. The value array x, label array y, and the above collection of image arrays then undergo conversion to numpy arrays along with encoding it using OneHot Categorical Encoding. This ensures that the datasets are properly formatted and ready for use in subsequent processing. After these steps are completed, the x and y arrays are split into training and testing datasets for further evaluation. In this study, first of all, the VGG16 model has been used to classify images into two categories: healthy or infected. The pixel value used for the three RGB channels was set to 224. The model was trained using the ImageNet weights, while the top layer of the model was omitted. There is no use in training existing weights. The output image was flattened and the softmax function was utilized as an activation function for the prediction of the two classes. Moreover, the model was trained for five epochs in this study using the adams optimizer and categorical cross-entropy as the loss function. The number of epochs can be manipulated as per requirements. The same steps have been followed for the other two models also, ResNet50 and InceptionV3.

2.1. *VGG16*

VGG stands for Visual Geometry Group. VGG16 is a deep convolutional neural network architecture that comprises 16 layers, including 13 convolutional layers and 3 fully connected layers. Small 3x3 convolutional filters are a distinctive feature of the VGG16 architecture. This feature makes it possible to extract more precise features from the input image. After the convolutional layers, max-pooling layers are added to reduce the spatial dimensionality of the feature maps. After that, fully connected layers are used to classify the output of the convolutional layers. The number of filters in each convolutional layer can be calculated using Equation 1 below:

$$2 * *n \tag{1}$$

where n is the index of the layer in the network. For instance, since the first convolutional layer's index is 1, it contains $2 * *1 = 2$ filters. The formula involved in calculating the output size from each convolution layer is given in Equation 2:

$$[(N - f)/S] + 1 \tag{2}$$

where N is the size of the input feature map, f is the size of the convolutional filter, and S is the stride of the convolution. ImageNet is a complex dataset that has around more than a million photos belonging to 1000 different object categories. VGG16 was initially used to showcase better performance. More complex structures like ResNet were developed due to the widespread adoption of VGG16 as the baseline model for various different computer vision applications. Fig. 2 shows the model architecture of VGG16.

2.2. *ResNet50*

ResNet stands for Residual Network. ResNet-50 model is a deep convolutional neural network with 50-layers. These 50-layers consist of convolutional layers, batch normalization layers, and fully connected layers. A shortcut connection is also present which skips over the convolutional layers. This linking enhances the data flow through the network and shortens the effects of vanishing gradient issues. ResNet50 is a type of artificial neural network that builds a network by piling residual blocks on top of one another. This architecture involves the approach of using residual blocks that allows a network to learn residual functions rather than mapping the input directly to the output. It was considered to be a novel approach to the problem of vanishing gradients in deep neural networks. The output of a residual block could be formulated as shown in Equation 3:

$$activation(W2*activation(W1*x+b1)+b2)+x \tag{3}$$

here, x is the input feature map, W1 and W2 are the weight matrices, b1, and b2 are the biases, and activation is the activation function (usually ReLU). The model architecture of ResNet50 is shown in Fig. 3.

2.3. *Inception V3*

InceptionV3 is a deep convolutional neural network architecture developed by Google to handle picture categorization tasks. The primary innovation in Inception v3 is the use of the Inception module, which allows the network to learn representations at different scales and dimensions. The Inception module consists of several convolutional layers with different filter sizes and pooling operations, followed by a concatenation of their output feature maps. This method allows the network to gather both fine and coarse characteristics from the input image. Calculation of the output of the Inception module can be expressed as in equation 4:

$$concatenate([x, y, z, a], axis = channel_axis) \tag{4}$$

here, x, y, z are $Conv2D\_1x1, Conv2D\_3x3$, and $Conv2D\_5x5$ convolutional layers with filter sizes of 1x1, 3x3, and 5x5 respectively, with input feature map. a is the MaxPooling2D, a pooling operation, and the axis specifies the axis along which the output feature maps are concatenated. The above architecture could be visualized as shown in Fig. 4:

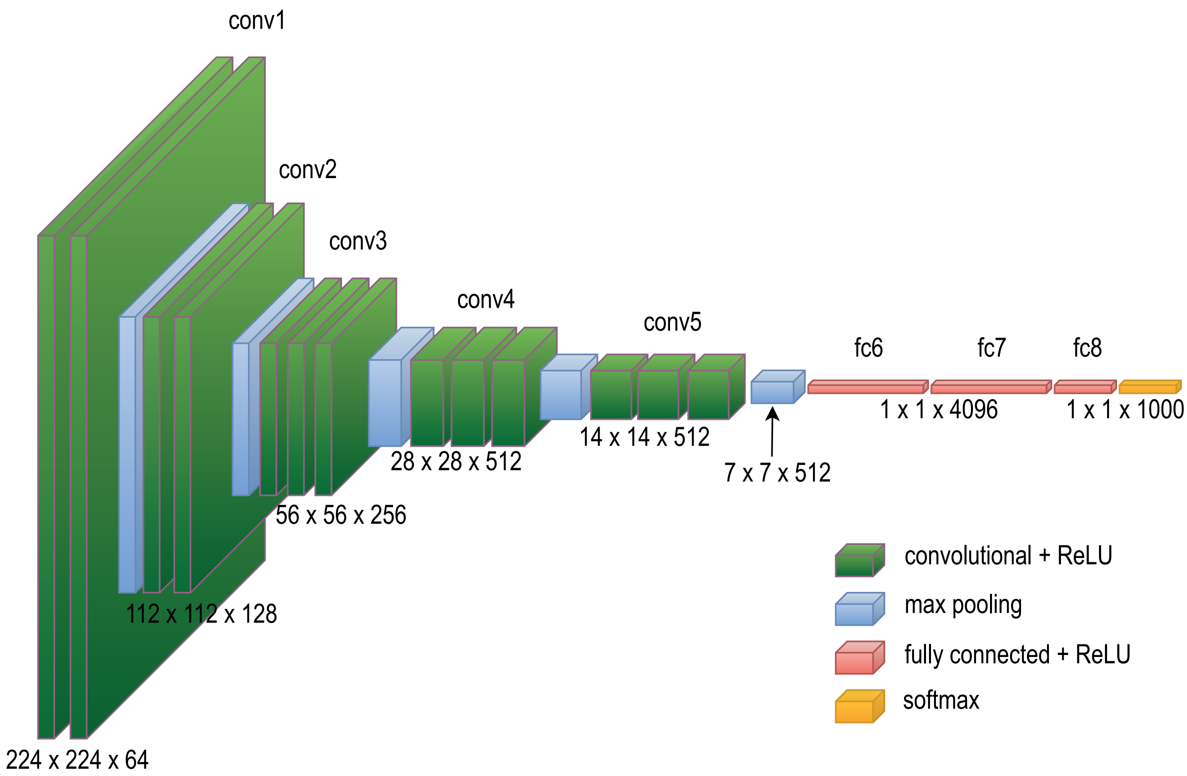


Figure 2. Model architecture for VGG16.

The primary goal of InceptionV3 is to consume less computing power by altering the Inception structures from earlier versions. Other methods to enhance the network's performance were also introduced in InceptionV3, such as batch normalization, factorised convolutions, and label smoothing.

2.4. *EfficientNet B7*

The EfficientNet is a convolutional neural network architecture that achieves cutting-edge accuracy on typical image classification and transfer learning tasks by scaling up the model network width through compound scaling. This method uses a combination of neural architecture search and scaling techniques to find the optimal balance between model size and accuracy. The efficientnet implementation in keras offers a variety of models, ranging from B0 to B7. This efficientnet returns the keras classification model that has previously been pre-trained with imagenet and optionally loaded with weights. 256-pixel pictures and imagenet weights were used to feed the efficientnetB7 model. It is flattened in the output layer before the prediction layer and a dense layer with sigmoid as the activation function is added. Adam serves as the model's optimizer, with loss represented by categorical crossentropy. The expression for scaling the depth of the network could be expressed as in equation 5:

$$depth = \alpha ** \phi \tag{5}$$

here, α(alpha) is a constant value, 0.43 for EfficientNet B7. It controls the amount of scaling. ϕ(phi) is a scaling factor that determines the size of the model. For EfficientNet B7, α is set to 0.43. The expression for scaling the width of the network could be expressed as in equation 6:

$$width = \beta ** \phi \tag{6}$$

here, β(beta) is a constant value, 0.25 for EfficientNet B7. It controls the amount of scaling. ϕ(phi) is a scaling factor that determines the size of the model. Also, the expression for scaling the resolution of the network is shown in Equation 7:

$$resolution = \gamma ** \Omega \tag{7}$$

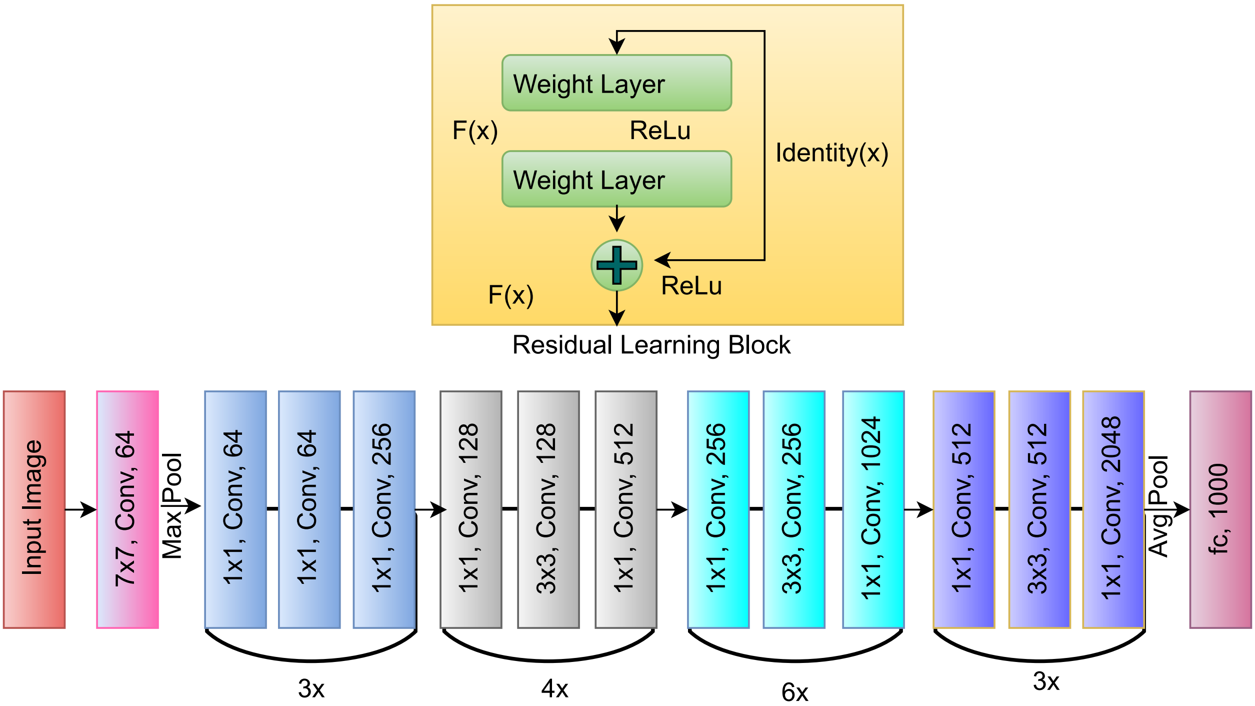


Figure 3. ResNet50 model architecture.

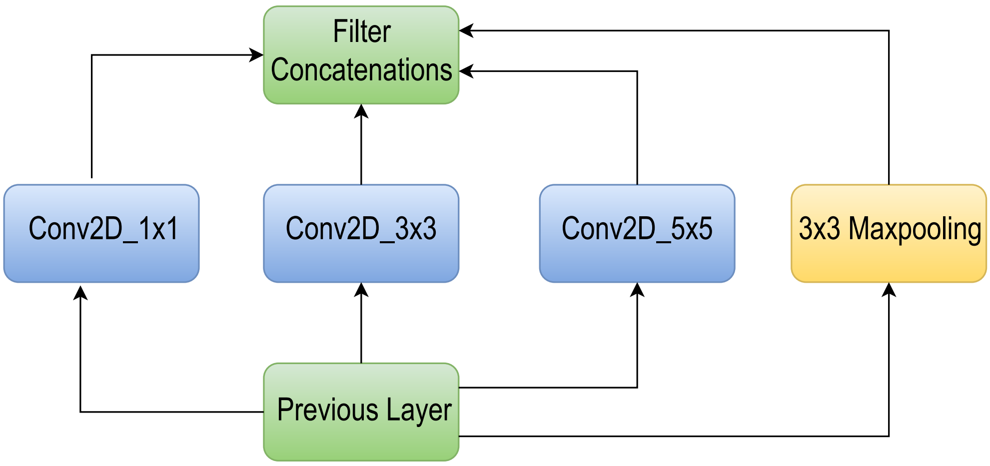


Figure 4. InceptionV3 model architecture.

here, γ(gamma) is a constant value, 2.0 for EfficientNet B7. It controls the amount of scaling. Ω(omega) is a scaling factor that determines the size of the model. Ω is set to 3 for EfficientNet B7.

2.5. *Xception*

Xception is a convolutional neural network architecture that uses maxpooling and depth-wise separable convolutions. It is a sort of convolutional operation that separates the common convolutional operation into the depth-wise convolution and the pointwise convolution steps. The model was initially created by Google researchers as an interpretation of the Inception model, but with depth-wise separable convolution, instead of ordinary convolution. The pointwise convolution uses 1x1 convolutions to mix the outputs from the depth-wise convolution across channels, whereas the depth-wise convolution applies a single filter to each input channel separately. In terms of computational time, depth-wise convolutions are more effective than ordinary convolutions. In this model, the data first passes through the entry flow, which consists of convolutional and separable convolutional layers, then flows to the middle layer, which includes separable convolutional layers for feature maps and is repeated eight times, before passing through the exit layer. Batch normalization comes after all convolutions and separable convolutions. With a lot fewer parameters than other models, Xception can achieve high accuracy thanks to this extreme variation of depthwise separable convolutions, or "Separable Convolutions".

2.6. *MobileNet V2*

The MobileNet architecture is appropriate for application in mobile devices with smaller models since it is a lightweight convolutional neural network and has an inverted residual structure. The design comprises two major blocks, each with three layers: a residual block with stride 1 and a shrinking block with stride 1. The blocks are divided into two layers: a depth-wise convolution on top of the first 1x1 convolution with ReLu. A combination of depth-wise separable convolutions and linear bottlenecks is used to attain high accuracy with a small number of parameters. Adding ReLu once more will give it the ability to function as a linear classifier on a non-zero volume portion of the domain output, the third layer is once more a 1x1 convolution, but without non-linearity. A model architecture for MobileNet is shown below in Fig. 4.

2.7. *DenseNet 121*

DenseNet is a convolutional neural network architecture that makes the connectivity pattern between layers simpler. Each layer in a DenseNet architecture is connected to every other layer directly in a feed-forward fashion, hence named as Densely Connected Convolutional Network. DenseNet121 is a deep neural network architecture that consists of 121 layers and has dense connectivity which leads to a high degree of parameter sharing and feature reuse, which improves the efficiency and accuracy of the network. The input image is initially processed through a convolutional layer in DenseNet121 before being sent through a max-pooling layer. Then, there are four dense blocks, each of which has a group of convolutional layers that extract characteristics from the input. There are transition layers in between the dense blocks that combine convolution and pooling procedures to lower the spatial dimensionality of the output feature maps. The final classification output is created by passing the output from the last dense block through a fully connected layer and a global average pooling layer. These steps could be visualized through the image shown in Fig. 6 below.

3. Results and Discussion

The dataset has been created by taking real-life images from different hospitals in Kolkata itself. The images are of various shapes and sizes because they are manually taken and went through pre-processing. To segment only the crucial area of interest for the categorization, we employed U-net architecture. The predicted mask generated by U-net along with the output for healthy images is shown in Fig. 8.

After the images were preprocessed, these images were fed to different classification deep learning models and attained remarkable accuracy for predicting the classification of conjunctivitis. We have used the Adam optimizer and loss function as Categorical Crossentropy for this task and the training accuracy and loss were calculated over 15 epochs. The accuracy and loss curves are plotted by averaging the scores in a particular epoch over all folds.

The different models were trained for 15 epochs and the loss and accuracy scores were learned through this process. Graphs of loss and accuracy for each model were plotted for the 15 epochs averaged over the epochs in the respective training and validation stages. We have done experimentation with different deep-learning architectures including VGG 16, ResNet 50, Inception V3, Xception, EfficientNet B7, MobileNet V2, and DenseNet 121, and have achieved respective training, testing, and validation accuracies. The loss and accuracy graphs for all the models are shown in Fig. 9 - 12.

As it is clear from Table 1 that we are getting the

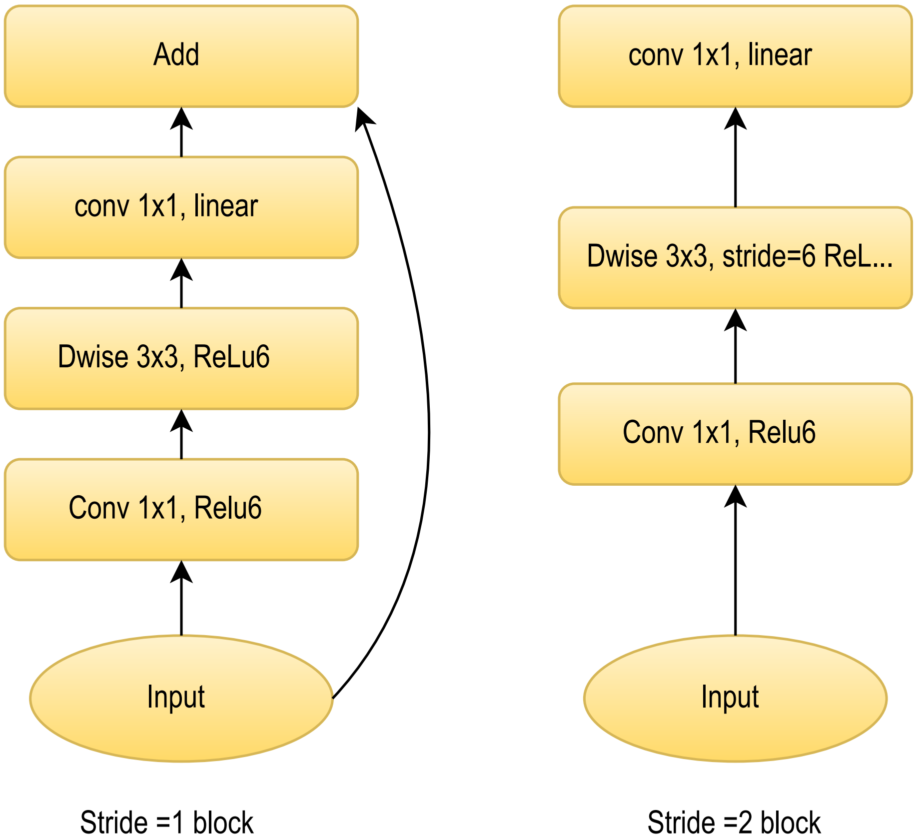


Figure 5. MobileNet V2 model architecture.

highest training accuracy of 98.25 %, testing accuracy of 87.25 %, and validation accuracy of 64.25% for the VGG 16 model, and considered the best-fit model for our experiment.

Table 1. Comparative study of the deep learning models

ID	Model Name	Training	Testing	Validation
1	VGG 16	98.25	87.25	64.25
2	ResNet 50	97	81	71.25
3	Inception V3	95	68.75	50
4	Xception	92	50	45
5	EfficientNet B7	87.5	1138	57.14
6	MobileNet V2	98	93	57.25
7	DenseNet 121	78	50.25	56.25

4. Conclusions

The classification task performed by the model successfully classifies the eye images as either healthy or infected and we have achieved a profound accuracy of 97.50% by our VGG 16 classification model. As mentioned in our previous paper, we have applied U-net architecture for automated segmentation and used that on several different models including ResNet 50, Inception V3, Xception, EfficientNet B7, MobileNet V2, DenseNet 121 with individual accuracy of 92.50%, 50%, 65%, 97.50%, 85% and 50% on the real-life data collected for our experiment.

Bibliography

1. Adak, M., Chatterjee, A., Roy, N.D. and Mahmud, M., 2023, February. i ConDet2: An Improved Conjunctivitis Detection Portable Healthcare App Powered by Artificial Intelligence. In Applied Intelligence and Informatics: Second International Conference, AII 2022, Reggio Calabria, Italy, September 1–3, 2022, Proceedings (pp. 205-218). Cham: Springer Nature Switzerland.
2. Holland, E.J., Fingeret, M. and Mah, F.S., 2019. Use of topical steroids in conjunctivitis: a review of the evidence. Cornea, 38(8), pp.1062-1067.

3. Fu, H., Cheng, J., Xu, Y. and Liu, J., 2019. Glaucoma detection based on deep learning network in fundus image. Deep learning and convolutional neural networks for medical imaging and clinical informatics, pp.119-137.
4. Soysa, A. and De Silva, D., 2020. A mobile base application for cataract and conjunctivitis detection. In Proceedings of ICACT (pp. 76-78).
5. Salducci, M. and La Torre, G., 2020. COVID-19 emergency in the cruise's ship: a case report of conjunctivitis. La Clinica Terapeutica, 171(3), pp.e189-e191.
6. Xia, J., Tong, J., Liu, M., Shen, Y. and Guo, D., 2020. Evaluation of coronavirus in tears and conjunctival secretions of patients with SARS-CoV-2 infection. Journal of medical virology, 92(6), pp.589-594.
7. Lai, T.H., Tang, E.W., Chau, S.K., Fung, K.S. and Li, K.K., 2020. Stepping up infection control measures in ophthalmology during the novel coronavirus outbreak: an experience from Hong Kong. Graefe's Archive for Clinical and Experimental Ophthalmology, 258, pp.1049-1055.
8. Ozturker, Z.K., 2021. Conjunctivitis as sole symptom of COVID-19: A case report and review of literature. European Journal of Ophthalmology, 31(2), pp.NP145-NP150.
9. Hu, Y., Chen, T., Liu, M., Zhang, L., Wang, F., Zhao, S., Liu, H., Xia, H., Wang, Y. and Li, L., 2020. Positive detection of SARS-CoV-2 combined HSV1 and HHV6B virus nucleic acid in tear and conjunctival secretions of a non-conjunctivitis COVID-19 patient with obstruction of common lacrimal duct. Acta Ophthalmologica, 98(8), pp.859-863.
10. Seah, I. and Agrawal, R., 2020. Can the coronavirus disease 2019 (COVID-19) affect the eyes? A review of coronaviruses and ocular implications in humans and animals. Ocular immunology and inflammation, 28(3), pp.391-395.

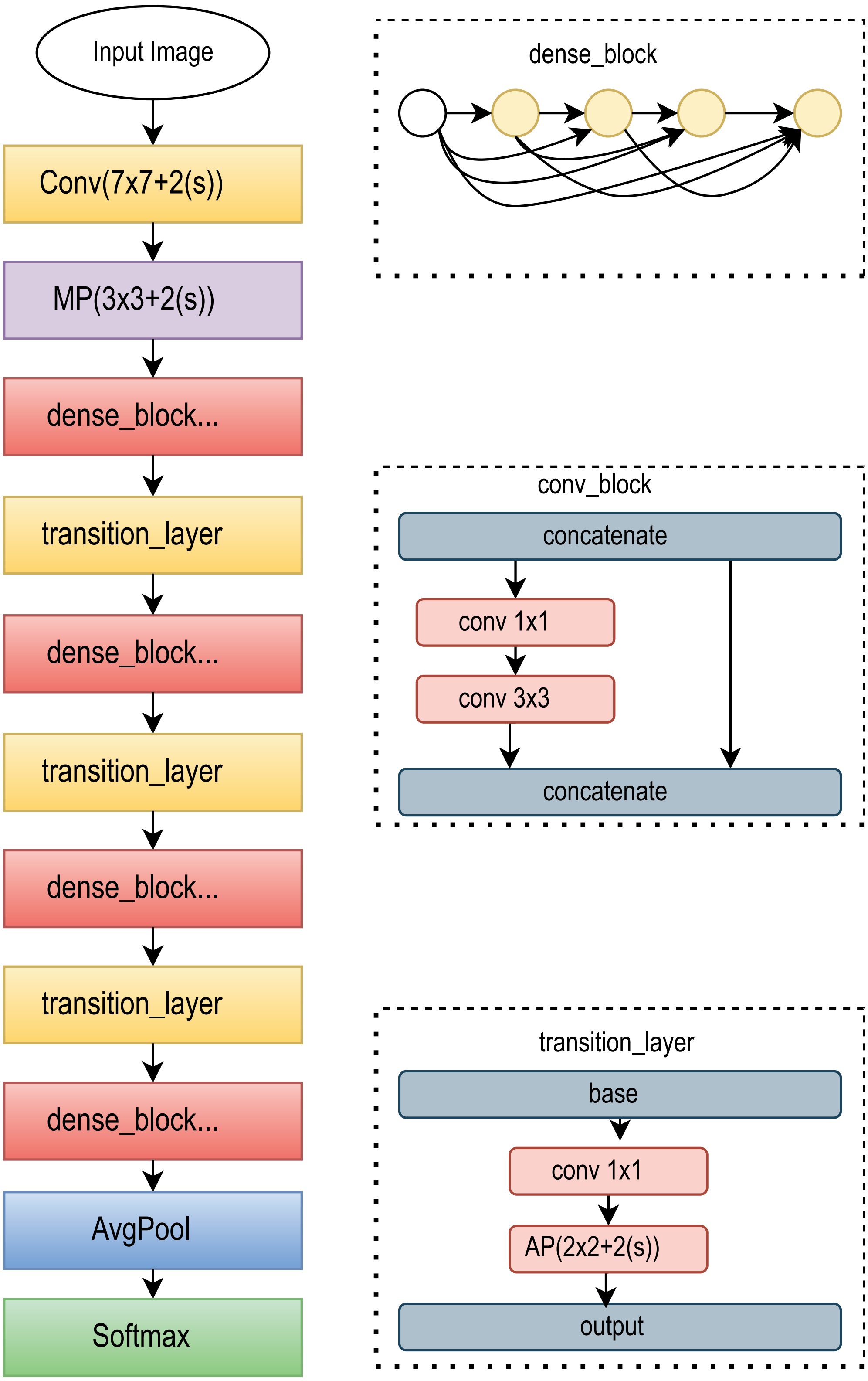


Figure 6. Model architecture for DenseNet121.

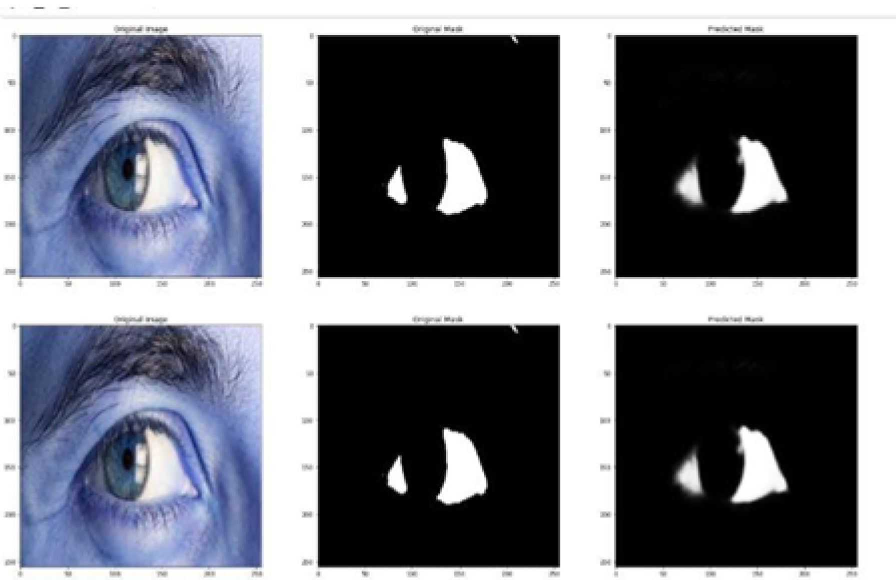


Figure 7. Predicted mask using U-net of Healthy eye for ground truth.

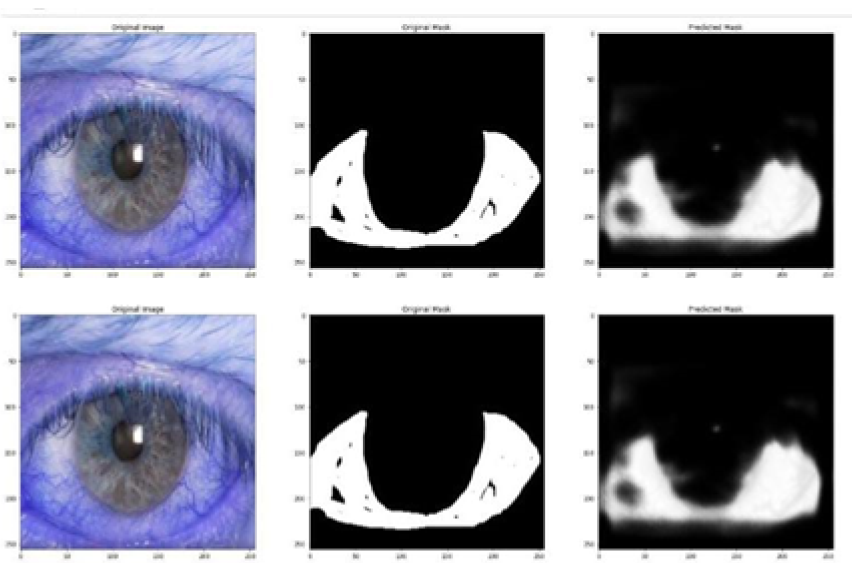
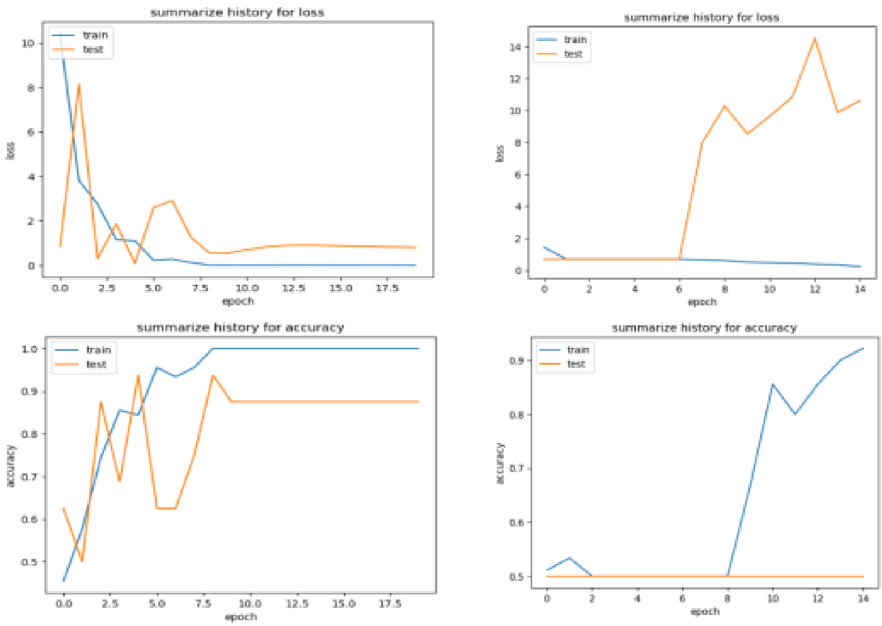


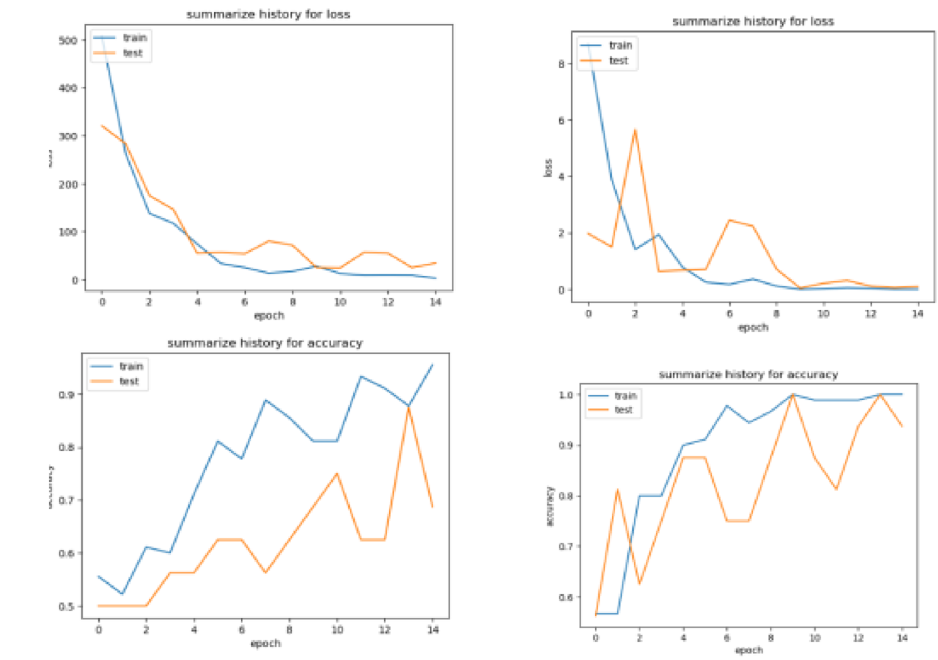
Figure 8. Predicted mask using U-net of Infected eye for ground truth.



(a) Loss and Accuracy graph of VGG 16

(b) Loss and Accuracy graph of Xception

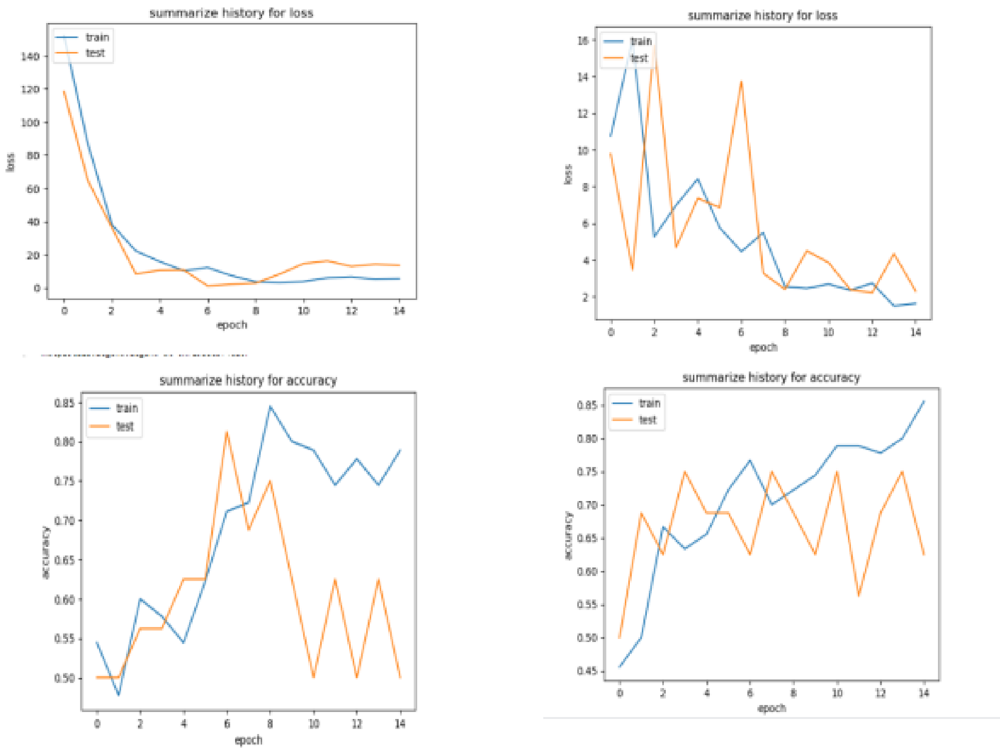
Figure 9. Loss and Accuracy graph of VGG 16 and Xception.



(c) Loss and Accuracy graph of Inception V3

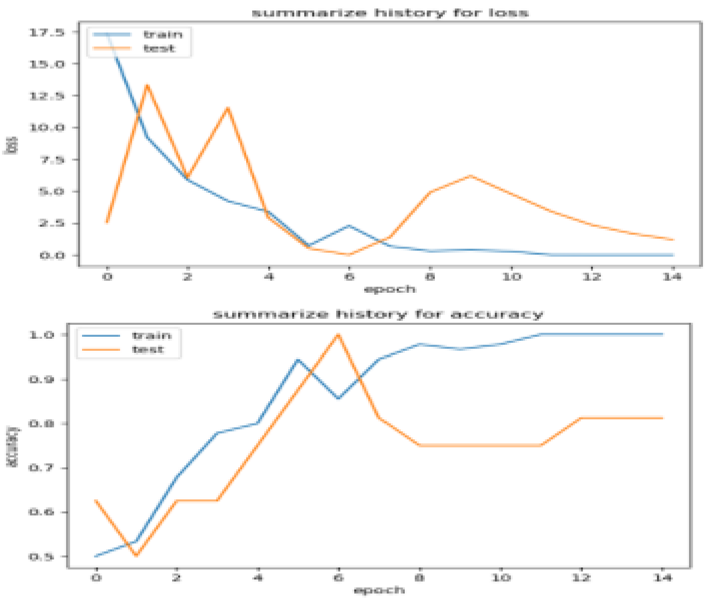
(d) Loss and Accuracy graph of MobileNet V2

Figure 10. Loss and Accuracy graph of f Inception V3 and MobileNet V2.



(e) Loss and Accuracy graph of DenseNet 121 (f) Loss and Accuracy graph of EfficientNet B7

Figure 11. f DenseNet 121 and EfficientNet B7.



(g) Loss and Accuracy graph of the best model ResNet 50

Figure 12. Loss and Accuracy graph of the best model ResNet 50.